

claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_4fd1a4ea-6c0\agent-ad0655a607fb32457.jsonl`
- SHA-256: `38a5821ec2a19dbc3c9bf3ad559255da3d482e38149eeae520af428ba2596cd`
- Source modified: `2026-07-02T09:31:18+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_4fd1a4ea-6c0`
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`
```

Transcript

1. user (2026-07-02T09:29:46.009Z)

ADVERSARIAL VERIFIER. A reviewer of the P23 main.py->orchestration.py extraction claims:

TITLE: caplog tests pin logger="backend.main" but the asserted warnings now come from logger "backend.orchestration"

SEVERITY: nit

DETAIL: tests/test_api_cloud_dispatch.py:311 and :444 use caplog.at_level("WARNING", logger="backend.main") and then assert on warnings that the extraction moved into backend/orchestration.py (the 'cloud video_id %s != local' warning at orchestration.py:286 and the 'ignoring unknown compute' warning at orchestration.py:209), which are now emitted under logger name "backend.orchestration". The tests still pass (I ran both: 2 passed) only because pytest's caplog handler captures at the root logger regardless of the logger= argument, and root is at INFO via main.py's basicConfig. The logger= pin is now vestigial and brittle ? if these tests ever tighten to filter records by record.name == "backend.main", or if root level handling changes, they break silently-later rather than here. Not a merge blocker; worth a one-line test fix (point at "backend.orchestration" or drop the logger= arg). Related cosmetic delta: server log lines for these messages change %(name)s from backend.main to backend.orchestration.

EVIDENCE: tests/test_api_cloud_dispatch.py:311, tests/test_api_cloud_dispatch.py:444; backend/orchestration.py:209, backend/orchestration.py:286 (verified: both tests pass in the worktree)

REFACTOR UNDER REVIEW (uncommitted, in worktree C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs, branch refactor/p23-orchestration-split off origin/master):

```
- NEW backend/orchestration.py (739 lines): 10 members moved VERBATIM from backend/main.py:
  _finalize_reingest, _report_config_for_input, _ingest_remote_segments, _cloud_available,
  _maybe_dispatch_remote, _execute_processing, _remote_input_size_bytes, _CompileError,
  _resolve_compile, _execute_compile.
- backend/main.py 2360 -> 1703 lines; re-exports all 10 names via "from backend.orchestration
import (...)" (noqa F401);
  5 now-unused imports removed (csv, math, Tuple, StitchingConfig, localize_watermark).
- Enumerated intentional deltas (everything else is AST-identical, machine-verified):
  (1) _maybe_dispatch_remote: "req: ProcessRequest" -> string forward-ref "ProcessRequest"
(TYPE_CHECKING import).
  (2) _execute_processing: inserted "import backend.main as _main" after docstring;
global_config/db_instance/
  ProcessRequest resolved via _main.<name> at call time (the job_worker seam convention).
  (3) two "# type: ignore[...]" comments (AST-invisible).
- NEW tests/test_main_module_budget.py: line budget 1750, route ceiling 18, re-export identity
check.
- Verified already: AST-identity (10 moved + 51 retained members), ruff+format clean, mypy green
(142 files),
  ratchet test passes. Full suite running separately.
```

Inspect with: `git -C "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs" diff` (and `git -C "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs" status`); the new files are untracked/modified in the worktree.

READ-ONLY: do not modify the worktree, do not checkout/commit.

Independently verify in the worktree (READ-ONLY). Default skeptical: `real=true` ONLY if you can reproduce the evidence AND the failure scenario is concrete (a test that would fail, a runtime path that breaks, an operational regression). A theoretical concern with no concrete consumer = `real=false`.

2. user (2026-07-02T09:29:54.595Z)

```
)
assert out["status"] == "failed"
assert "could not read the result" in out["message"]
assert db.query_play_segments(out["video_id"], {}) == []

def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
    """If the worker re-hashes different bytes (result.video_id != local id), ingest still
    proceeds
    under the LOCAL id but logs a WARNING so the orphaned-report condition is diagnosable."""
    db, _bucket, monkeypatch = cloud_env
    _inject_fake(monkeypatch, _FakeCloudClient(video_id="DIVERGED"))
    with caplog.at_level("WARNING", logger="backend.main"):
        out = m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
    assert out["status"] == "success"
    assert len(db.query_play_segments(out["video_id"], {})) == 2
    assert any(
        "DIVERGED" in r.getMessage() and "!= local" in r.getMessage()
        for r in caplog.records
    )
    assert "isn't configured" in out["message"]

def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
    """An unrecognised compute value is ignored (logged) and the server default applies ? here a
    cloud_run server still dispatches to the cloud."""
    db, _bucket, monkeypatch = cloud_env
    fake = _FakeCloudClient()
    _inject_fake(monkeypatch, fake)
    with caplog.at_level("WARNING", logger="backend.main"):
        out = m._execute_processing(
            m.global_config,
            m.db_instance,
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="banana"
            ),
        )
    assert out["status"] == "success"
    assert len(fake.calls) == 1 # fell back to the configured cloud_run default
    assert any("unknown compute" in r.getMessage() for r in caplog.records)
```

3. user (2026-07-02T09:30:00.912Z)

```
        ran,
    )

    # SPA compute-picker (item 3): a per-request choice overrides the server's configured
    default.
    # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch (guarded by
```

```

# _cloud_available so we fail clearly rather than silently running local); None / unknown ?
default.
choice = (req.compute or "").strip().lower()
if choice and choice not in ("local", "cloud"):
    logger.warning(
        "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
        req.compute,
    )
    choice = ""
if choice == "local":
    return None # explicit "run on my machine" ? in-process regardless of the server's
target
if choice == "cloud":
if result.returncode != 0:
    return _failed(f"cloud job failed (returncode {result.returncode}).", True)
if result.video_id and result.video_id != video_id:
    # The worker re-hashes the bucket bytes; a divergence means the input changed between
this
    # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
    logger.warning(
        "[API] cloud video_id %s != local %s ? input bytes differ between the API hash "
        "and the worker hash; ingesting under the local id.",
        result.video_id,
        video_id,
    )

    notify("ingesting (cloud_run)")
backend/orchestration.py:43:logger = logging.getLogger(__name__)
backend/main.py:33:logger = logging.getLogger(__name__)

```

4. user (2026-07-02T09:30:21.244Z)

```

backend/orchestration.py:209:         "[API] ignoring unknown compute=%r (expected
'local'/'cloud'); using server default.",
backend/orchestration.py:286:         "[API] cloud video_id %s != local %s ? input bytes
differ between the API hash "
28:logging.basicConfig(

```

5. user (2026-07-02T09:30:28.249Z)

```

.. [100%]
2 passed in 2.65s

```

6. user (2026-07-02T09:30:44.995Z)

```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\scratchpad\wt-
docs\tests\test_api_cloud_dispatch.py:311:     with caplog.at_level("WARNING",
logger="backend.main"):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\scratchpad\wt-
docs\tests\test_api_cloud_dispatch.py:444:     with caplog.at_level("WARNING",
logger="backend.main"):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\scratchpad\wt-docs\backend\ui\assets\index-
ukuS9wxa.js:49:[Omitted long matching line]

```

7. user (2026-07-02T09:31:18.359Z)

Structured output provided successfully